

实验题目：_____ 模块参数及 Proc 文件系统

一、 实验内容

按实验内容编写驱动程序

编译驱动程序

在 Ubuntu 上加载驱动程序并利用 Proc 文件系统的读写进行驱动程序测试

写一个简单的字符设备驱动程序，要求：

定义一个整型模块参数 N，定义一个全局 char 指针 buffer_ptr；

在加载函数中使用 kmalloc 函数申请大小为 N 的空间，buffer_ptr 指向该空间；

在卸载函数中使用 kfree 函数释放该空间；

通过读 proc 文件系统对该 buffer 进行读取；

通过写 proc 文件系统对该 buffer 进行赋值；

模块参数 N 在加载过程中指定，proc 文件的名称可自定义。

注意事项：

本实验只考察 proc 文件系统和模块参数，因此可以不注册常规的设备文件和设备操作函数。

在编写驱动程序及测试程序时，在关键位置增加打印信息，驱动程序打印调用 printk 函数

申请空间调用 kmalloc 函数，释放空间调用 kfree 函数

通过 Proc 文件系统写 buffer 时，注意 buffer 大小判断，防止写入过多数据导致内存访问崩溃

在 ubuntu 虚拟机上完成程序验证

二、 实验原理（代码）

seq_demo_drv2.c:

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/uaccess.h>
#include <linux/slab.h>

static struct mutex lock;
static int N = 0;
static char *buffer_ptr = NULL;

// 定义模块参数 N
module_param(N, int, S_IRUGO | S_IWUSR);
MODULE_PARM_DESC(N, "Size of the buffer");

// 写操作函数
static ssize_t _seq_write(struct file *file, const char __user * buffer, size_t count, loff_t *ppos){
    size_t to_copy;          //记录需要写多少

    printk(KERN_INFO "proc_write: Enter with count=%zu, pos=%lld\n", count, *ppos);

    mutex_lock(&lock);
    if (!buffer_ptr || N <= 0) {
        mutex_unlock(&lock);
        printk(KERN_ERR "proc_write: Buffer not available or invalid size\n");
        return -EINVAL;
    }

    if(*ppos >= N){
```

```

        mutex_unlock(&lock);
        printk(KERN_INFO "proc_write: Position %lld beyond buffer size %d\n", *ppos, N);
        // 当位置超出缓冲区时，视为成功但不进行实际写入操作
        // 这样 echo 命令可以正常结束
        return count;}
//防止溢出的操作，在当前要写入的数量和申请内存的余量中取小
to_copy = min(count, (size_t)(N - *ppos));
printk(KERN_INFO "proc_write: Will copy %zu bytes\n", to_copy);

if(copy_from_user(buffer_ptr + *ppos, buffer, to_copy)){
    mutex_unlock(&lock);
    printk(KERN_ERR "proc_write: Failed to copy from user\n");
    return -EFAULT;}
//移动指针
*ppos += to_copy;
printk(KERN_INFO "proc_write: Copied %zu bytes, new pos=%lld\n", to_copy, *ppos);

mutex_unlock(&lock);
return to_copy;}

// 顺序文件操作相关函数
static void *_seq_start(struct seq_file *m, loff_t *pos) {
    printk(KERN_INFO "proc_seq_start: pos=%lld\n", *pos);
    mutex_lock(&lock);
    return (*pos == 0) ? (void *)1 : NULL;}

static void *_seq_next(struct seq_file *m, void *p, loff_t *pos) {
    (*pos)++;
    printk(KERN_INFO "proc_seq_next: new pos=%lld\n", *pos);
    return NULL;}

static void _seq_stop(struct seq_file *m, void *p) {
    printk(KERN_INFO "proc_seq_stop\n");
    mutex_unlock(&lock);}

static int _seq_show(struct seq_file *m, void *v)
{
    int i;
    unsigned char c;

    printk(KERN_INFO "proc_seq_show: Enter\n");

    if (!buffer_ptr) {
        printk(KERN_ERR "proc_seq_show: Buffer not available\n");
        return -EINVAL;}

    seq_printf(m, "Buffer[%d]: ", N);

```

```

    for (i = 0; i < N; ++i) {
        c = buffer_ptr[i];
        if (c >= 32 && c <= 126) { // 可打印 ASCII 字符
            seq_putc(m, c);
        } else if (c == 0) { // 终止字符
            break;
        } else { // 非打印字符转义
            seq_printf(m, "\\x%02x", c);}}
    seq_putc(m, '\n');
    printk(KERN_INFO "proc_seq_show: Displayed %d bytes\n", i);
    return 0;}

static struct seq_operations _seq_ops = {
    .start  = _seq_start,
    .next   = _seq_next,
    .stop   = _seq_stop,
    .show   = _seq_show};

static int _seq_open(struct inode *inode, struct file *file) {
    printk(KERN_INFO "proc_open: Enter\n");
    file->f_pos = 0;
    return seq_open(file, &_seq_ops);
}

static struct file_operations _seq_fops = {
    .open    = _seq_open, // 自定义打开函数
    .read    = seq_read,  // 标准读取函数，会调用 seq_ops 中的函数
    .write   = _seq_write, // 自定义写入函数
    .llseek  = seq_lseek,
    .release = seq_release};

static int __init init(void) {
    struct proc_dir_entry *entry;

    printk(KERN_INFO "Loading module with buffer size N=%d\n", N);
    if (N <= 0) {
        printk(KERN_ERR "Invalid buffer size %d\n", N);
        return -EINVAL;}

    // 初始化互斥锁
    mutex_init(&lock);

    // 分配内存
    printk(KERN_INFO "Allocating %d bytes for buffer\n", N);
    buffer_ptr = kmalloc(N, GFP_KERNEL);
    if (!buffer_ptr) {
        printk(KERN_ERR "Failed to allocate memory\n");

```

```

        return -ENOMEM;
    }

    // 初始化缓冲区为全 0
    memset(buffer_ptr, 0, N);
    printk(KERN_INFO "Buffer initialized to zero\n");

    // 创建 proc 文件
    entry = proc_create("my_buffer", S_IRUGO | S_IWUSR, NULL, &_seq_fops);
    if (!entry) {
        printk(KERN_ERR "Failed to create proc entry\n");
        kfree(buffer_ptr);
        return -ENOMEM;
    }

    printk(KERN_INFO "Module loaded successfully\n");
    return 0;
}

static void __exit fini(void) {
    printk(KERN_INFO "Unloading module\n");

    // 移除 proc 文件
    remove_proc_entry("my_buffer", NULL);
    printk(KERN_INFO "Proc entry removed\n");

    // 释放内存
    if (buffer_ptr) {
        printk(KERN_INFO "Freeing buffer memory\n");
        kfree(buffer_ptr);
        buffer_ptr = NULL;
    }

    // 销毁互斥锁
    mutex_destroy(&lock);

    printk(KERN_INFO "Module unloaded successfully\n");
}

module_init(init);
module_exit(fini);
MODULE_LICENSE("GPL");

```

single_demo_drv2.c:

```

#include <linux/init.h>
#include <linux/module.h>
#include <linux/types.h>

```

```

#include <linux/slab.h>
#include <linux/uaccess.h>
#include <linux/fs.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/mm.h>
MODULE_LICENSE("GPL");

/* integer module variable N */
static int N;
module_param(N, int, 0);
MODULE_PARM_DESC(N, "integer variable");

char* buffer_ptr = NULL;

//show function
static int lab2_proc_show(struct seq_file *m, void *v){
    if(buffer_ptr != NULL)
    {
        seq_printf(m, "%s\n", buffer_ptr);
    }
    return 0;
}

//open function
static int lab2_proc_open(struct inode *inode, struct file *file) {
    return single_open(file, lab2_proc_show, NULL);
}

//write function
static ssize_t lab2_proc_write(struct file *file, const char __user *buf, size_t count, loff_t *ppos)
{
    size_t writeBytes = count;

    if(count > (size_t)N)
        writeBytes = (size_t)N;

    copy_from_user(buffer_ptr, buf, writeBytes);
    return writeBytes;
}

static const struct file_operations lab2_file_ops = {
    .owner      = THIS_MODULE,
    .open       = lab2_proc_open,
    .read       = seq_read,
    .write      = lab2_proc_write,
    .llseek     = seq_lseek,

```

```

        .release      = single_release,
};

static int __init lab2_module_init(void) {
    printk("module init\n");
    buffer_ptr = (char*)kmalloc((size_t)N, GFP_KERNEL);
    proc_create_data("lab2", 0644, NULL, &lab2_file_ops, buffer_ptr); /* create /proc/lab2 */
    return 0;
}

static void __exit lab2_module_exit(void)
{
    printk("module exit\n");
    kfree(buffer_ptr);
    buffer_ptr = NULL;
    remove_proc_entry("lab2", NULL);
}

module_init(lab2_module_init);
module_exit(lab2_module_exit);

```

三、 实验结果及其分析

编译成功后，加载驱动程序，传入模块参数 N（此处测试传入 N=5）从而指定 buffer_ptr 指向空间的大小，最后卸载驱动；

第一次写入：12345，cat 也为 12345

第二次写入：123456789，cat 仍然为 12345

第三次写入：Yaoya，cat 为 Yaoya

证明驱动程序写入逻辑正确，也能处理一定的溢出情况

```

root@ubuntu:linux# cd test2
root@ubuntu:test2# make clean
rm -f *.ko *.mod.c *.o -core.depend
root@ubuntu:test2# make
make -C /lib/modules/3.2.14/build M=/home/linux/test2 modules
make[1]: 正在进入目录 `/usr/src/linux-source-3.2.0'
  CC [M]  /home/linux/test2/demo_drv2.o
Building modules, stage 2.
MODPOST 1 modules
  CC      /home/linux/test2/demo_drv2.mod.o
  LD [M]  /home/linux/test2/demo_drv2.ko
make[1]:正在离开目录 `/usr/src/linux-source-3.2.0'
root@ubuntu:test2# insmod demo_drv2.ko N=5
root@ubuntu:test2# echo -n "12345" > /proc/my_buffer
root@ubuntu:test2# cat /proc/my_buffer
Buffer[5]: 12345
root@ubuntu:test2# echo -n "123456789" > /proc/my_buffer
root@ubuntu:test2# cat /proc/my_buffer
Buffer[5]: 12345
root@ubuntu:test2# echo -n "Yaoya" > /proc/my_buffer
root@ubuntu:test2# cat /proc/my_buffer
Buffer[5]: Yaoya
root@ubuntu:test2# rmmod demo_drv2

```

以上操作打印的日志内容如下

```
[ 1854.411972] Loading module with buffer size N=5
[ 1854.411975] Allocating 5 bytes for buffer
[ 1854.411977] Buffer initialized to zero
[ 1854.412010] Module loaded successfully
[ 1893.852616] proc_open: Enter
[ 1893.852664] proc_write: Enter with count=5, pos=0
[ 1893.852666] proc_write: Will copy 5 bytes
[ 1893.852668] proc_write: Copied 5 bytes, new pos=5
[ 1903.096398] proc_open: Enter
[ 1903.096420] proc_seq_start: pos=0
[ 1903.096421] proc_seq_show: Enter
[ 1903.096424] proc_seq_show: Displayed 5 bytes
[ 1903.096426] proc_seq_next: new pos=1
[ 1903.096427] proc_seq_stop
[ 1903.096439] proc_seq_start: pos=1
[ 1903.096440] proc_seq_stop
[ 1913.776433] proc_open: Enter
[ 1913.776453] proc_write: Enter with count=9, pos=0
[ 1913.776455] proc_write: Will copy 5 bytes
[ 1913.776457] proc_write: Copied 5 bytes, new pos=5
[ 1913.776459] proc_write: Enter with count=4, pos=5
[ 1913.776461] proc_write: Position 5 beyond buffer size 5
[ 1915.832330] proc_open: Enter
[ 1915.832352] proc_seq_start: pos=0
[ 1915.832353] proc_seq_show: Enter
[ 1915.832356] proc_seq_show: Displayed 5 bytes
[ 1915.832358] proc_seq_next: new pos=1
[ 1915.832359] proc_seq_stop
[ 1915.832371] proc_seq_start: pos=1
[ 1915.832373] proc_seq_stop
```

```
[ 1932.038583] proc_open: Enter
[ 1932.038602] proc_write: Enter with count=5, pos=0
[ 1932.038604] proc_write: Will copy 5 bytes
[ 1932.038606] proc_write: Copied 5 bytes, new pos=5
[ 1934.242540] proc_open: Enter
[ 1934.242561] proc_seq_start: pos=0
[ 1934.242563] proc_seq_show: Enter
[ 1934.242565] proc_seq_show: Displayed 5 bytes
[ 1934.242567] proc_seq_next: new pos=1
[ 1934.242568] proc_seq_stop
[ 1934.242583] proc_seq_start: pos=1
[ 1934.242584] proc_seq_stop
[ 2004.757724] Unloading module
[ 2004.757732] Proc entry removed
[ 2004.757734] Freeing buffer memory
[ 2004.757735] Module unloaded successfully
```